Assignment 1: Git & GitHub Fundamentals

Report

- ***Full Name:*** Malak Lamrhaimima
- ***Student ID:*** 149131
- ***Course Code & Section:*** CSC 3324 01
- ***GitHub Username:*** malaklamrhaimima

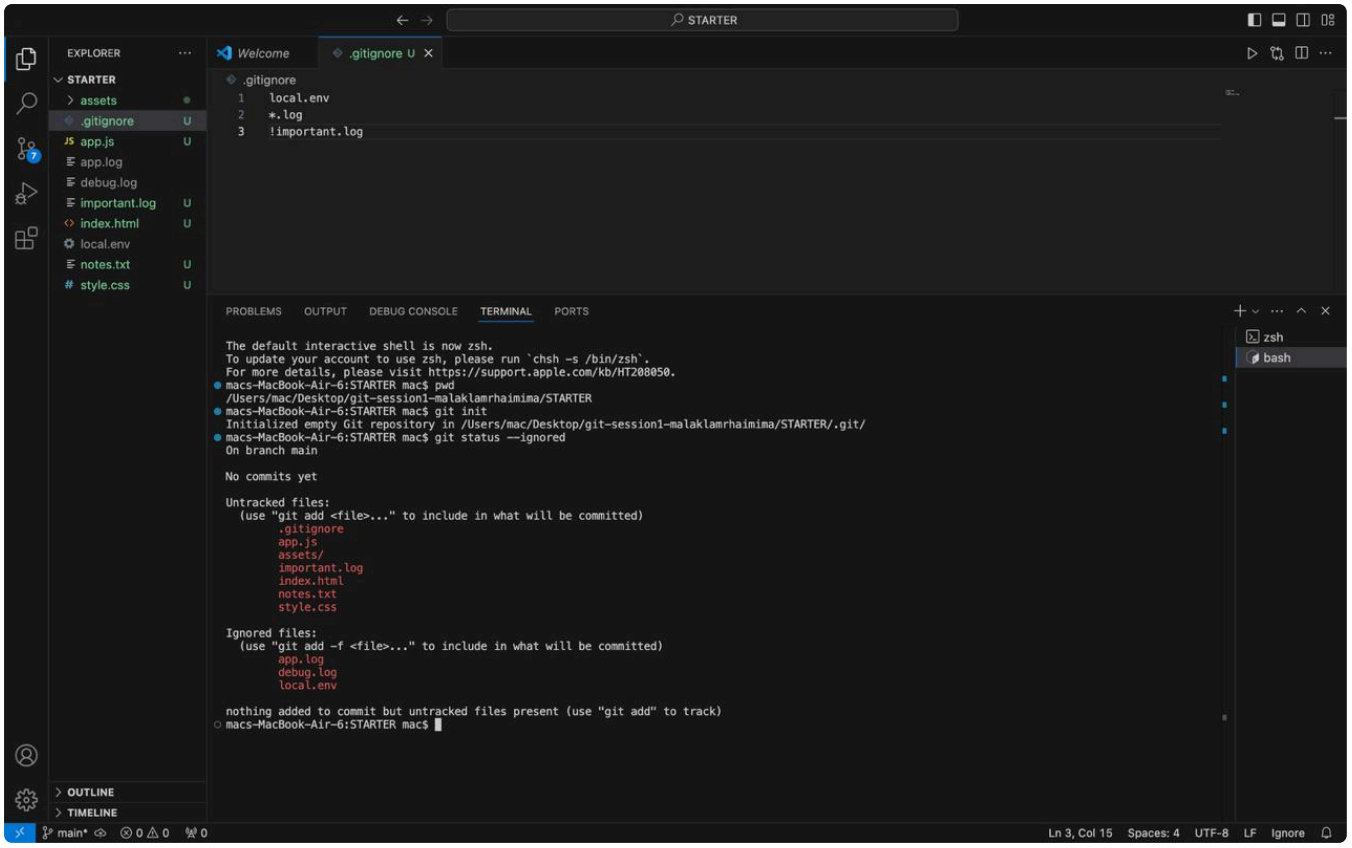
1. Repository Setup

I put the starter files in a folder on my Desktop called `git-session1-malaklamrhaimima` and opened it in VS Code on my Mac. After checking where I was with `pwd`, I ran `git init`, which created an empty repo on the `main` branch.

My `.gitignore` has three rules:

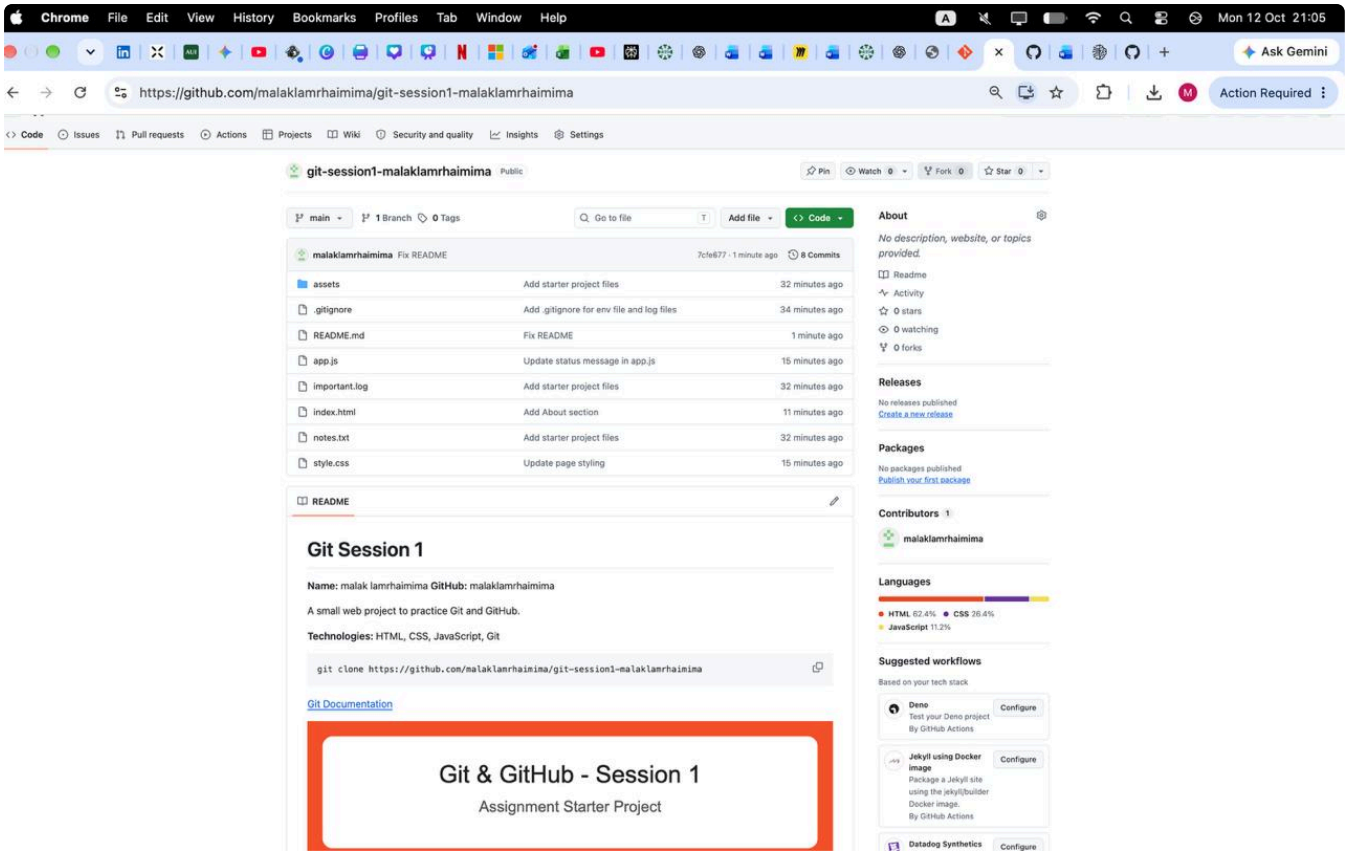
- `local.env`: my local settings file, which should not go online
- `*.log`: all log files, so `app.log` and `debug.log` are skipped
- `!important.log`: the one log file I still want Git to track

Running `git status --ignored` confirmed it: `app.log`, `debug.log` and `local.env` show up as ignored, and `important.log` is with the normal untracked files.



2. README & Commit History

The README shows my name and GitHub username, one line about the project, the technologies (HTML, CSS, JavaScript, Git), a code block with the `git clone` command, a link to the Git docs, and the banner image from the `assets` folder. Here is how GitHub displays it:

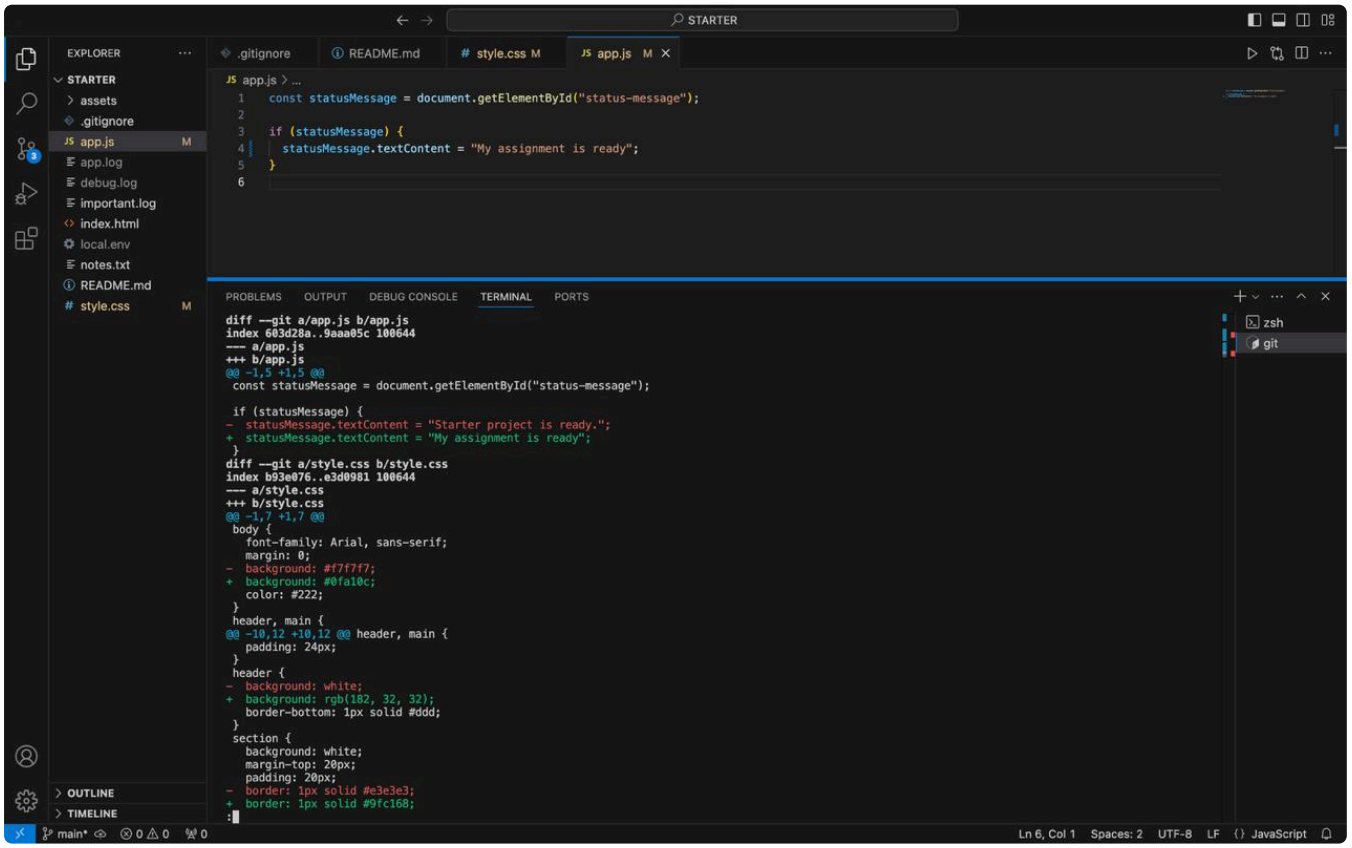


Each commit covers one step, and the message says what that step was:

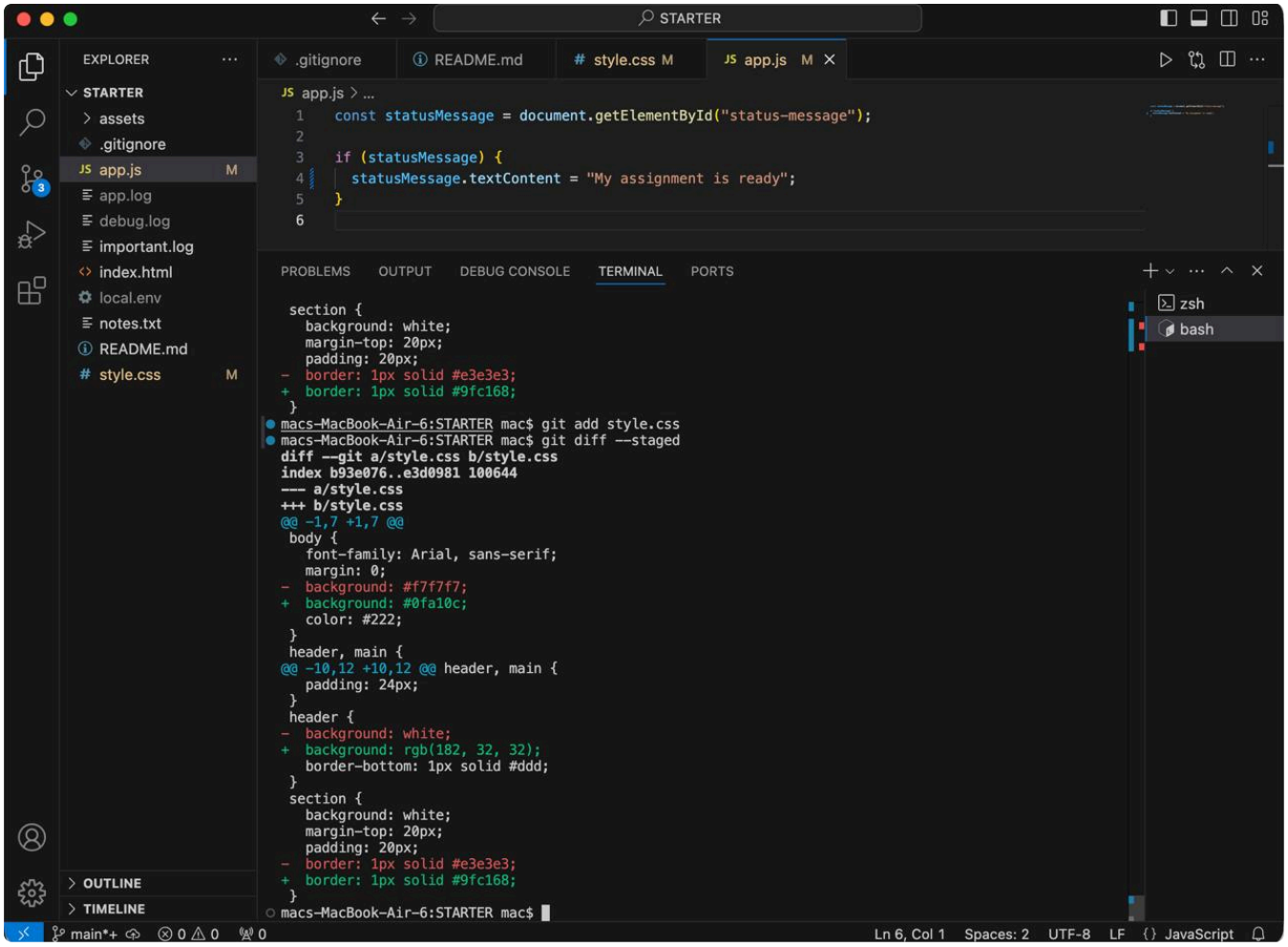
Commit	Message	Files
ab3054f	Add .gitignore for env file and log files	`.gitignore`
ef6fb3f	Add starter project files	starter HTML, CSS, JS, assets
4a03d5c	Add README with project info and banner	`README.md`
73e75b3	Update page styling	`style.css`
a4d1332	Update status message in app.js	`app.js`
97aecb7	Add About section	`index.html`
78d34ff	Merge feature/about into main	merge
7cfe677	Fix README	`README.md`
3149981	Update README.md	`README.md` (edited on GitHub)

3. Selective Staging

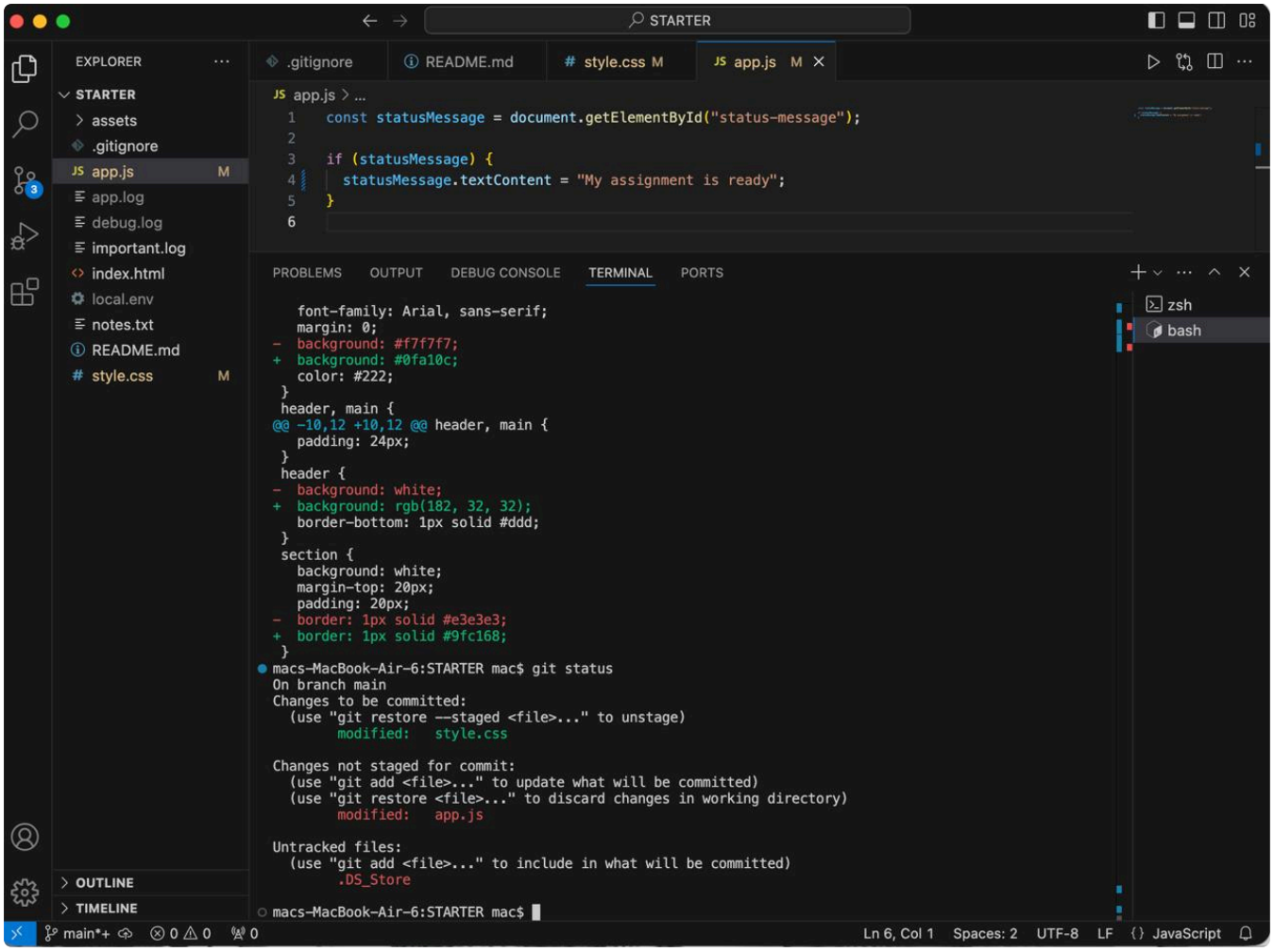
I edited `style.css` (new green background, red header, green border) and `app.js` (status text changed to "My assignment is ready"). `git diff` listed both files as changed but not staged.



Next I ran `git add style.css` on its own. `git diff --staged` only showed the CSS lines:



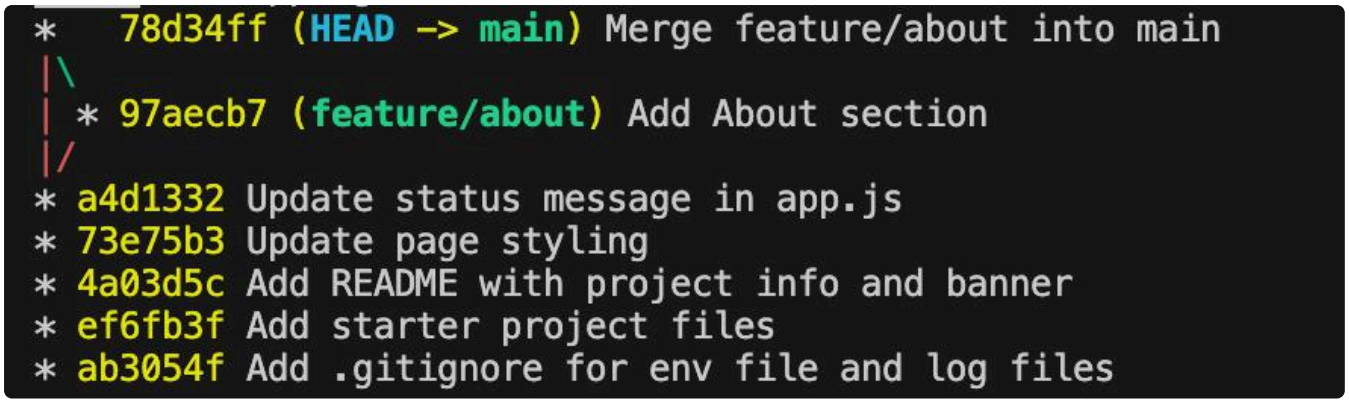
`git status` then showed `style.css` under "Changes to be committed" and `app.js` under "Changes not staged for commit". I committed the CSS as "Update page styling" and left `app.js` modified, then committed it afterwards as its own step.



4. Branch & Merge

For the About feature I created a branch called `feature/about` with `git switch -c feature/about`. On it I added an About section to `index.html` and committed it (97aecb7). Then I switched back with `git switch main` and merged using `git merge --no-ff feature/about`.

The `--no-ff` option forces a merge commit (78d34ff), so the history graph still shows where the branch split off and came back.



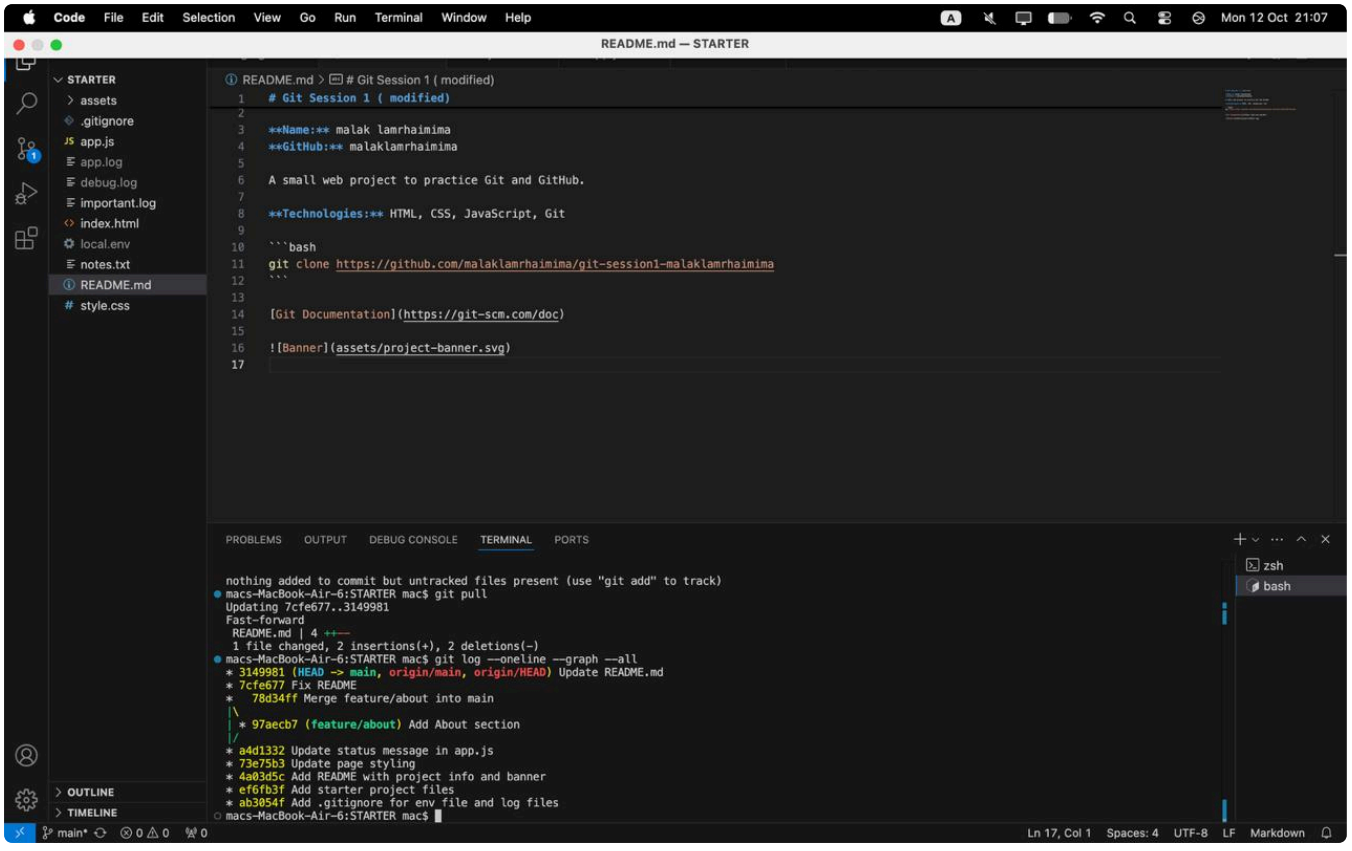
5. Remote Changes

I created an empty public repo on GitHub, added it as `origin` with `git remote add origin <url>`, and published my branch with `git push -u origin main`.

- **Commit vs push:** a commit only records the change in my local repo. Push is what sends my commits up to GitHub.
- **Fetch vs pull:** fetch gets the latest commits from GitHub into `origin/main` without changing my files. Pull does a fetch and then merges, so my files are updated.

To test it I edited `README.md` on the GitHub website. After `git fetch`, `git status` told me my branch was 1 commit behind `origin/main`, and my local README was still the old one.

Then `git pull` fast-forwarded `main` from 7cfe677 to 3149981, and the log shows `main` and `origin/main` on the same commit.



6. Fair Use of AI

I used AI when I ran into problems with the commands on my Mac. The terminal kept showing a message about zsh being the default shell, an extra `.DS_Store` file appeared as untracked, and `git diff` opened a screen I didn't know how to leave. I asked what

these meant and how to get past them. I typed every command myself, and I checked each answer by running `git status` and `git log` to make sure the result matched what the assignment asked for.

7. Sources & Reflection

Sources

- Session 1 slides and practice files
- Git documentation
- gitignore documentation

Problems I fixed

- macOS creates a hidden `.DS_Store` file that showed up as untracked. It is not part of the project, so I left it out of my commits.
- The terminal said "The default interactive shell is now zsh". It is just a macOS notice, and Git works the same in bash or zsh.
- `git diff` opened a pager ending with `:`. Pressing `q` gets back to the terminal.

Reflection

The hardest concept for me was the difference between fetch and pull, because after fetching GitHub was ahead but my files did not change. If Git does something unexpected, I would first run `git status` to check my branch and what is staged, and then `git log --oneline --graph --all` to see where `main` and `origin/main` are.